# AgroSmart

Análisis de Algoritmos y Estructuras de Datos

*Módulo: Registro de Cultivos*

## 1. Búsqueda Lineal

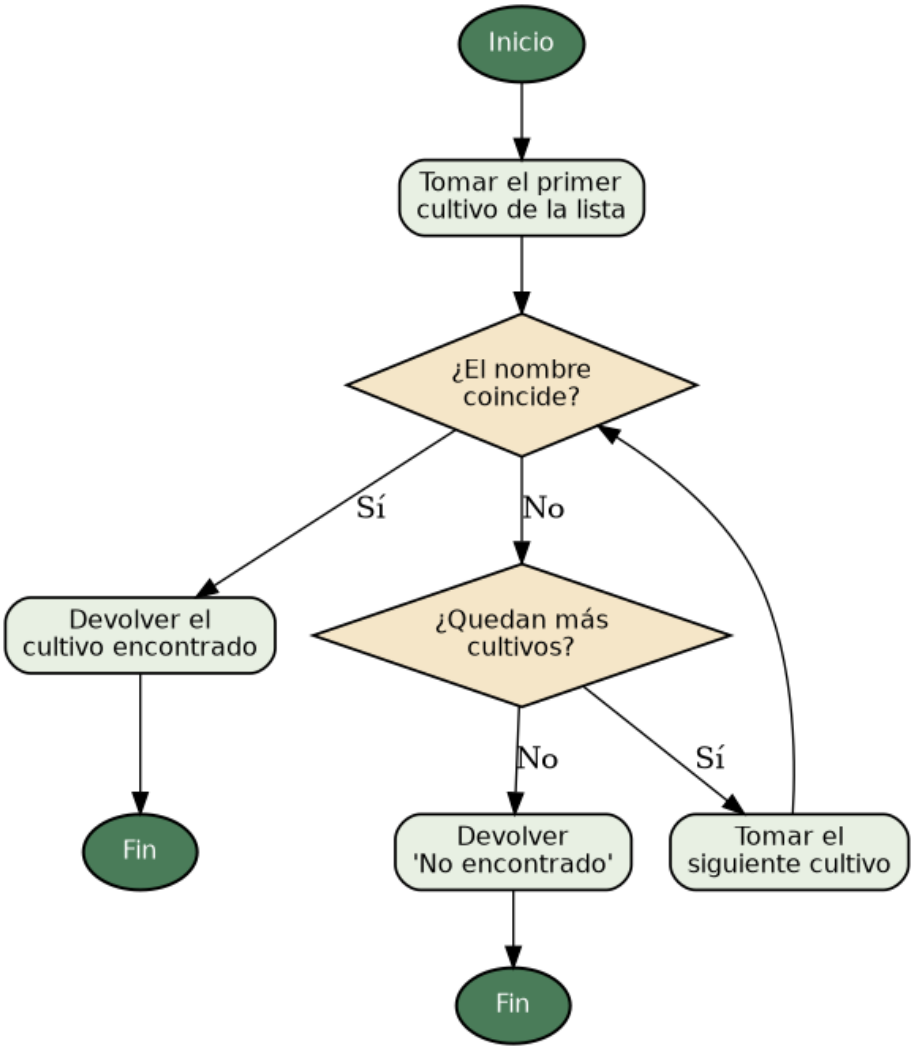
### Explicación del funcionamiento

La búsqueda lineal recorre la lista de cultivos uno por uno, comparando el nombre de cada cultivo con el nombre que se está buscando. Si encuentra una coincidencia, devuelve ese cultivo de inmediato. Si llega al final de la lista sin encontrar nada, informa que no existe. Es el método de búsqueda más simple y no requiere que la lista esté ordenada.

### Pseudocódigo

```
FUNCION buscar_por_nombre(lista, nombre_buscado)
  PARA CADA cultivo EN lista
    SI cultivo.nombre == nombre_buscado ENTONCES
      DEVOLVER cultivo
    FIN SI
  FIN PARA
  DEVOLVER "No encontrado"
FIN FUNCION
```

### Diagrama de flujo




## Análisis de complejidad

| | |
|---|---|
| **Complejidad temporal** | O(n) — en el peor caso (el cultivo buscado está al final, o no existe), se revisan los números de cultivos de la lista. |
| **Complejidad espacial** | O(1) — no se necesita memoria adicional más allá de la lista original. |

## Justificación de su elección

Se eligió la búsqueda lineal porque es sencilla de implementar, no requiere que los datos estén ordenados previamente, y es adecuada para la cantidad de cultivos que maneja un sistema agrícola de este tamaño (decenas o cientos de registros, no miles). Además, sirve como punto de comparación frente a la búsqueda binaria.

# 2. Búsqueda Binaria

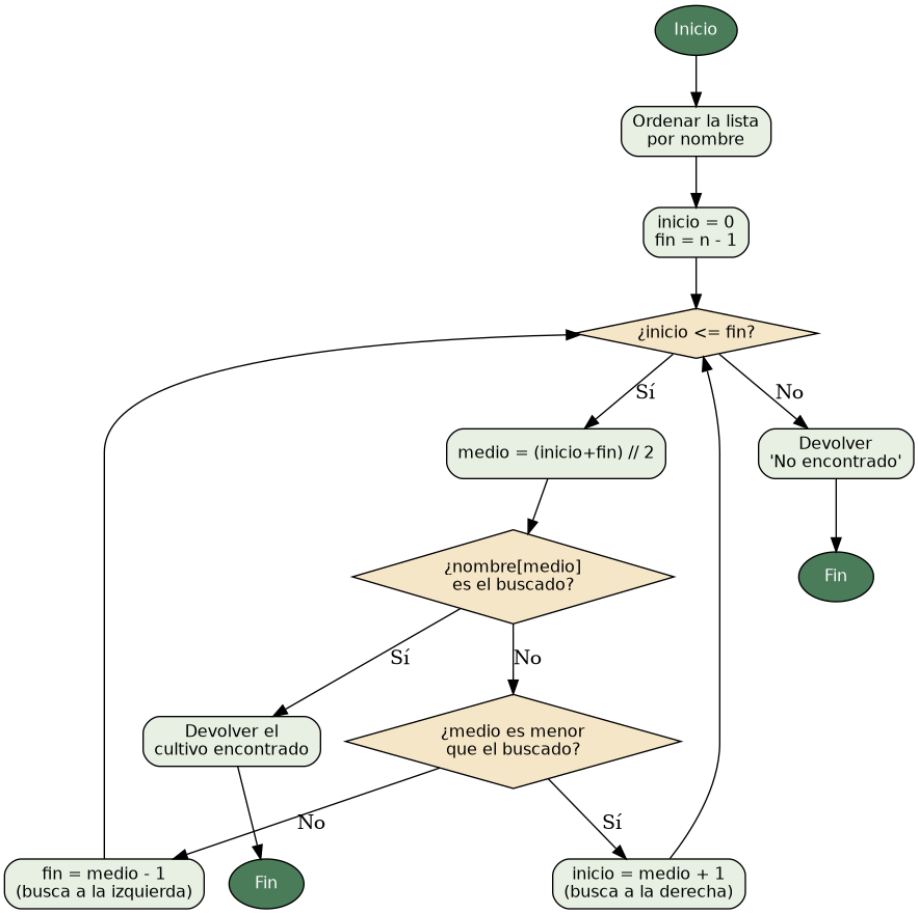
## Explicación del funcionamiento

La búsqueda binaria primero ordena una copia de la lista alfabéticamente por nombre. Luego, en cada paso, revisa el elemento que está justo en la mitad del rango de búsqueda: si coincide, lo devuelve; si el nombre buscado es alfabéticamente mayor, descarta toda la mitad izquierda; si es menor, descarta toda la mitad derecha. Este proceso se repite, reduciendo a la mitad el espacio de búsqueda en cada paso, hasta encontrar el cultivo o quedarse sin rango.

## Pseudocódigo

```
FUNCION buscar_por_nombre_binaria(lista, nombre_buscado)
  lista_ordenada = ORDENAR(lista, por nombre)
  inicio = 0
  fin = LONGITUD(lista_ordenada) - 1
  MIENTRAS inicio <= fin
    medio = (inicio + fin) DIV 2
    SI lista_ordenada[medio].nombre == nombre_buscado ENTONCES
      DEVOLVER lista_ordenada[medio]
    SINO SI lista_ordenada[medio].nombre < nombre_buscado ENTONCES
      inicio = medio + 1
    SINO
      fin = medio - 1
    FIN SI
  FIN MIENTRAS
  DEVOLVER "No encontrado"
FIN FUNCION
```

## Diagrama de flujo




## Análisis de complejidad

| | |
|---|---|
| **Complejidad temporal** | O(log n) — en cada paso se descarta la mitad de los elementos restantes, por lo que el número de comparaciones crece muy lentamente incluso con muchos cultivos. |
| **Complejidad espacial** | O(n) — se crea una copia ordenada de la lista para no alterar el orden original (que puede estar ordenado por otro criterio, como la fecha). |

## Justificación de su elección

Se eligió como segundo algoritmo de búsqueda porque permite comparar directamente su rendimiento contra la búsqueda lineal: mientras la lineal revisa hasta n cultivos, la binaria necesita como máximo log2(n). Su única desventaja es que requiere ordenar los datos primero, lo cual es aceptable dado que ya se cuenta con un algoritmo de ordenamiento en el sistema.

# 3. Tabla Hash

## Explicación del funcionamiento

La Tabla Hash es una estructura de datos que permite guardar y encontrar información de forma casi instantánea, usando una 'función hash' que convierte una llave (en este caso, el código del cultivo, como "CULT001") en un número de casillero. Al guardar un dato, se calcula su casillero y se coloca ahí. Al buscarlo, se vuelve a calcular el mismo casillero y se revisa directamente ahí, sin necesidad de recorrer toda la estructura. Cuando dos llaves distintas caen en el mismo casillero (una colisión), ambas se guardan en una lista dentro de ese casillero.

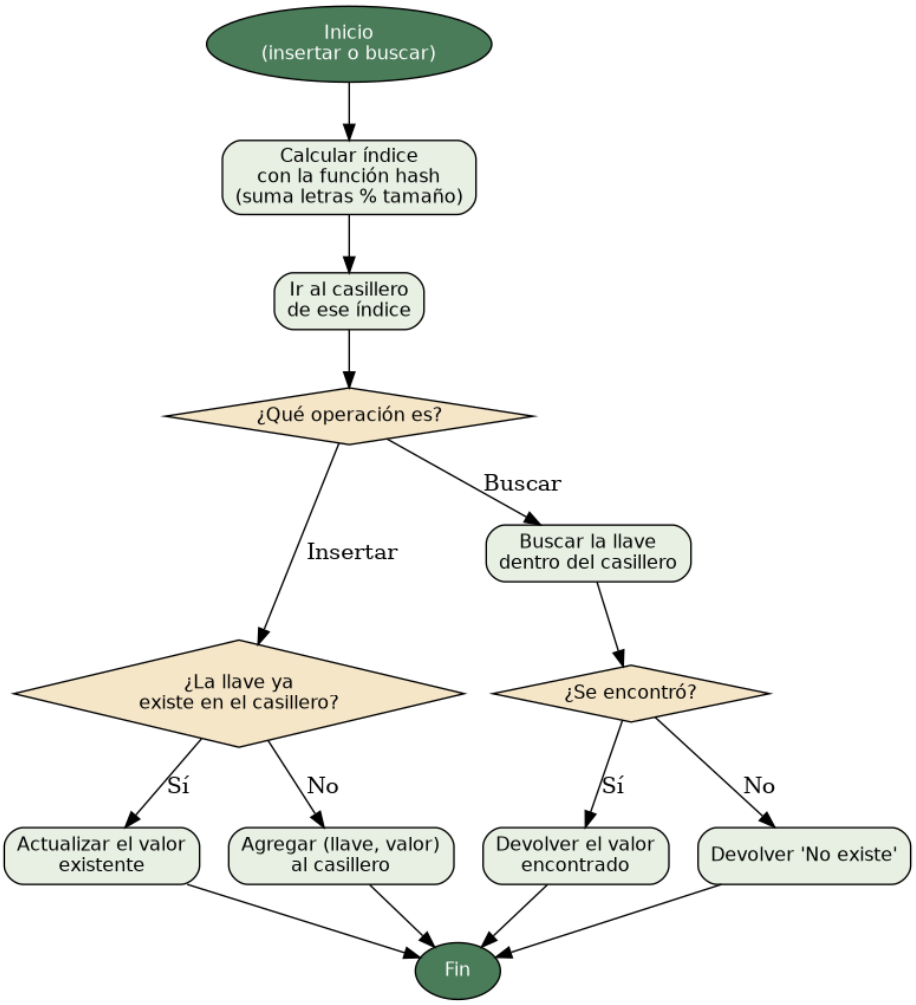
## Pseudocódigo

```
FUNCION calcular_indice(llave, tamano)
  suma = SUMA(valor_ascii(caracter) PARA CADA caracter EN llave)
  DEVOLVER suma MODULO tamano
FIN FUNCION

FUNCION insertar(llave, valor)
  indice = calcular_indice(llave, tamano)
  casillero = casilleros[indice]
  SI llave YA EXISTE en casillero ENTONCES
    ACTUALIZAR su valor
  SINO
    AGREGAR (llave, valor) al casillero
  FIN SI
FIN FUNCION

FUNCION buscar(llave)
  indice = calcular_indice(llave, tamano)
  casillero = casilleros[indice]
  PARA CADA (llave_guardada, valor) EN casillero
    SI llave_guardada == llave ENTONCES
      DEVOLVER valor
    FIN SI
  FIN PARA
  DEVOLVER "No existe"
FIN FUNCION
```

## Diagrama de flujo




## Análisis de complejidad

| | |
|---|---|
| **Complejidad temporal** | O(1) en el caso promedio — se calcula el casillero directamente y normalmente solo hay 0 o 1 elementos ahí. En el peor caso (muchas colisiones en el mismo casillero), sería O(n). |
| **Complejidad espacial** | O(n) — se guarda una copia de cada cultivo dentro de la tabla, además de la lista original. |

## Justificación de su elección

Se implementó una Tabla Hash propia (en lugar de usar directamente un diccionario de Python) para poder explicar con claridad su funcionamiento interno. Es la estructura ideal para buscar un cultivo por su código único, ya que ofrece acceso prácticamente instantáneo sin importar cuántos cultivos existan, a diferencia de la búsqueda lineal o binaria.

# 4. Ordenamiento por Inserción

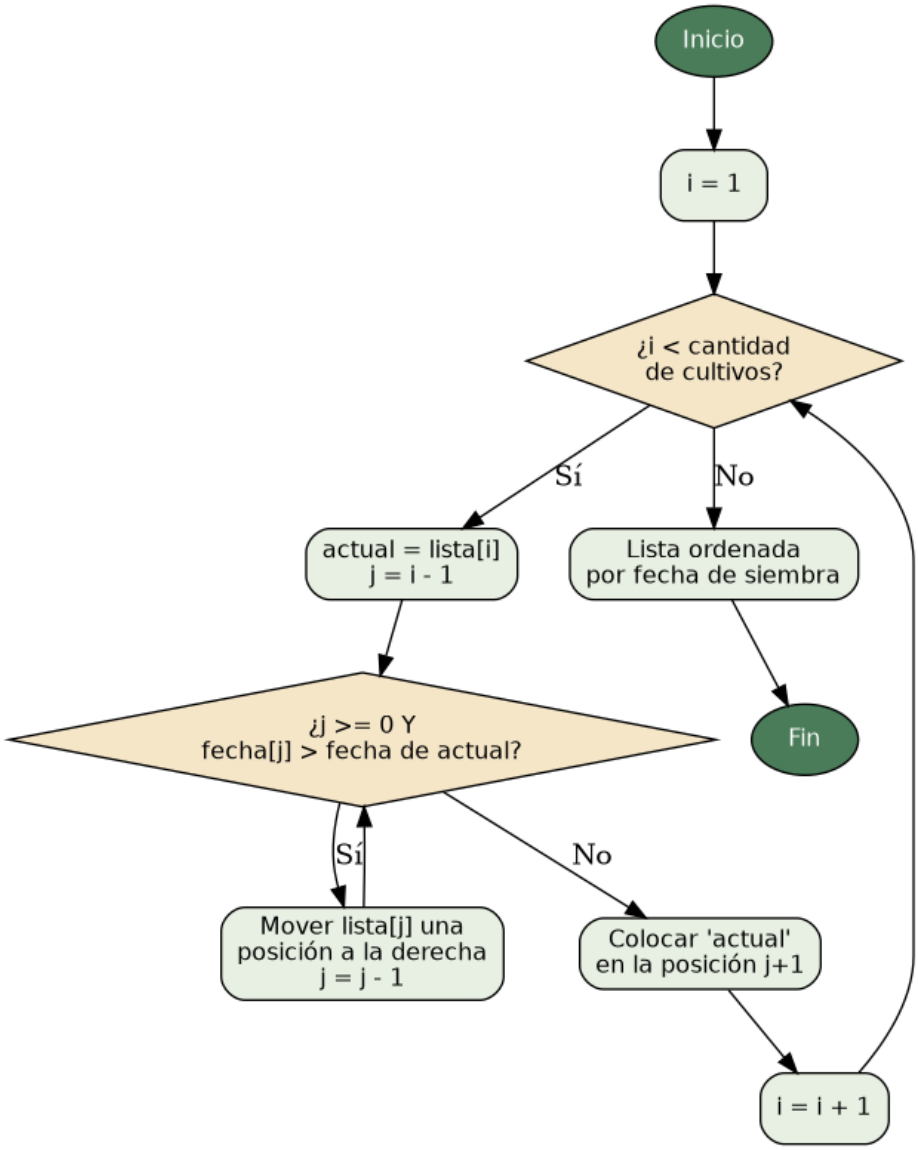
## Explicación del funcionamiento

El ordenamiento por inserción funciona de forma parecida a como una persona ordena cartas en su mano: toma un elemento a la vez y lo va colocando en la posición correcta entre los elementos que ya están ordenados, comparándolo hacia atrás y corriendo los demás elementos un puesto a la derecha cuando hace falta. Este algoritmo se usa para ordenar los cultivos por su fecha de siembra, de la más antigua a la más reciente.

## Pseudocódigo

```
FUNCION ordenar_por_fecha_siembra(lista)
  PARA i DESDE 1 HASTA LONGITUD(lista) - 1
    actual = lista[i]
    j = i - 1
    MIENTRAS j >= 0 Y lista[j].fecha_siembra > actual.fecha_siembra
      lista[j + 1] = lista[j]
      j = j - 1
    FIN MIENTRAS
    lista[j + 1] = actual
  FIN PARA
FIN FUNCION
```

## Diagrama de flujo




## Análisis de complejidad

| | |
|---|---|
| **Complejidad temporal** | O(n²) en el peor caso — si la lista está en orden inverso, cada elemento debe compararse con todos los anteriores. |
| **Complejidad espacial** | O(1) — el ordenamiento se hace "en el mismo lugar" (in-place), sin necesitar una lista adicional. |

## Justificación de su elección

Se eligió el ordenamiento por inserción por ser fácil de entender y explicar, y porque su rendimiento es aceptable para la cantidad de cultivos que maneja un sistema como este (no miles de registros). Además, es un buen punto de comparación frente a algoritmos de ordenamiento más avanzados que puedan implementar otros módulos del proyecto, como Merge Sort o Quick Sort.